Compiling Extended LAMBDA into Pure Lambda Calculus

Church data, delayed control flow, and recursion

Koji Aquilino

CS413 · Assignment 6 · Midterm project

Source and target languages

Source: the supplied Assignment 5 AST

Naturals and booleans · arithmetic and comparisons
Functions and application · let · pairs · conditionals · fix

Target: three immutable node forms

```
Term ::= Var(x) | Lam(x, body) | App(fn, arg)
```

Generated programs compute entirely through lambda reduction

Compiler architecture

Source AST → Compiler → Pure term → Evaluator → Observer → Data

Module	Responsibility
source_ast / source_ops	Supplied syntax, validation, printing
target	Immutable terms, names, substitution
compiler / encodings	Visitor translation and closed library terms
evaluator	One engine for source and target
observers	Bounded pure contexts for readable data
examples / demo	14 cases, source comparison, reporting

Church data

```
NUM(2) = lam s. lam z. s (s z)
```

A numeral repeats a supplied operation

```
TRUE  = lam t. lam f. t
FALSE = lam t. lam f. f
```

```
PAIR = lam a. lam b. lam p. p a b
```

```
FST p = p TRUE        SND p = p FALSE
```

Every library term is closed with no FVs; all names expand into pure syntax

Predecessor carries a pair

Each update: (previous, current) → (current, current + 1)

Updates	Previous	Current
0	0	0
1	0	1
2	1	2
3	2	3

After k updates:
previous = max(k − 1, 0)
current = k

```
PRED n = FST (n UPDATE (PAIR 0 0))
```

Arithmetic and comparisons

Operation	Encoded rule
Addition	ADD m n s z = m s (n s z)
Multiplication	MUL m n s z = m (n s) z
Truncated subtraction	SUB m n = n PRED m
Less than or equal	LE m n = ISZERO (SUB m n)
Equality	EQ m n = AND (LE m n) (LE n m)

Other comparisons follow by reversing arguments or negating

Compositional visitor translation

Source	Pure target construction
lam x. e	lam x. C[e]
e1 e2	C[e1] C[e2]
let x = init in body	(lam x. C[body]) C[init]
a + b	ADD C[a] C[b]
pair a b	PAIR C[a] C[b]

Compilation builds syntax; it never evaluates the program

Thunks preserve selective control flow

A thunk wraps a computation in a function to delay it

```
(C[c] (lam u. C[t]) (lam v. C[f])) I
```

1 Evaluate c and select one branch function
2 Apply that function to I to run its body
3 Leave the unused branch body untouched

```
false and LOOP → false        true or LOOP → true
```

Names and recursive scope

Reserve every source name before generating binders

Fresh names use base_counter with one deterministic counter
Unused source binders are reserved too

```
fix f(x). body → Z (lam f. lam x. C[body])
```

The inner parameter wins when both names coincide

```
(fix f(f). f) 3 → 3
```

Z supplies delayed recursion

```
F = lam f. lam x. C[body]

Z = lam g.
      (lam x. g (lam v. x x v))
      (lam x. g (lam v. x x v))
```

lam v. x x v is a value; recursion resumes when v arrives

Z delays recursive calls; thunks skip the unselected branch

One weak call-by-value evaluator

Evaluate the function, then the argument, then apply

A lambda is a completed value; its body stays untouched
Source closures retain their defining environments
Target application uses capture-avoiding substitution

A last permitted step may complete successfully

Budget exhaustion means more work is needed; it is not a termination proof

Observation has its own budget

Execution returns a function; observation extracts readable data

```
Natural: value STEP BASE
STEP = lam tail. lam c. lam z. c tail
BASE = lam c. lam z. z
```

```
Boolean: value I FALSE → a distinct selected marker
```

Pairs: project and observe recursively, sharing one observation cap

The caller supplies the kind: NUM(0) and FALSE share a representation

Recursive factorial walkthrough

```
fact(n) = if n == 0 then 1
          else n * fact(n - 1)
```

```
C[fix fact(n). body] = Z (lam fact. lam n. C[body])
```

Calls: 4 → 3 → 2 → 1 → 0

Returns: 1 → 1 → 2 → 6 → 24

Source result = observed target result = 24

Execution: 471 steps Observation: 333 steps

Separate budgets: 10,000 steps each

Measured demonstration results

Example	Result	Execution steps	Observation steps
2 + 3 * 4	14	4	26
if true then 1 else LOOP	1	3	3
(lam x. 0) LOOP	Exhausted	10,000	Skipped
Factorial 4	24	471	333
Fibonacci 5	5	1,794	45
LOOP	Exhausted	10,000	Skipped

14/14 matched expectations · independent 10,000-step caps

Automated test evidence

213 tests pass

Scope: capture hazards, shadowing, open inputs, coincident fix binders

Evaluation: order, exact budgets, strict and skipped divergence

Encodings: operators, truth tables, recursion, nested pairs

Boundaries: pure output, unchanged inputs, compilation never evaluates

Fresh-process CLI checks compare all 14 saved terms with compiler output

Correctness claim and limitations

Closed, well-typed source programs returning finite data

With sufficient resources, observing the compiled result
should yield the same natural, boolean, or finite pair data

Bindings, strictness, selected branches, and recursion must agree

No static type checker or parser
Substitution may duplicate terms; recursive traversals can hit host limits
Step caps do not bound memory or elapsed time; tests are not a proof

Conclusion

Pure functions for data, control flow, and recursion

Church encodings represent the data

Thunks and Z control when computation unfolds

Observers expose readable results

14/14 required outcomes matched · 213 tests passed

Sources and AI attribution: DESIGN.md · README.md · AI-TRANSCRIPT.md